

JobHunt Project

Complete Frontend & Backend Overview — Easy Learning Guide

1. Project at a Glance

JobHunt is a React + TypeScript frontend connected to a Node.js + Express + MongoDB backend. The frontend displays pages/forms; the API layer sends requests; Express handles business logic; MongoDB stores data.

User → React Frontend → API Service → Express Backend → MongoDB → Response → React UI

Layer	Technology	Purpose
Frontend	React + TypeScript	Pages, components, forms and UI
Styling	Tailwind CSS + CSS	Layout, colors, spacing and theme
Routing	React Router	URL-to-page navigation
Charts	Recharts	Dashboard/analytics charts
API	TypeScript	Frontend/backend communication
Backend	Node.js + Express	REST APIs and business logic
Database	MongoDB	Candidate/program data
Uploads	Multer + XLSX	Excel/CSV and file processing

2. Frontend Folder Structure

```
src/
  ├── api/
  │   ├── candidate.ts
  │   ├── candidates.js
  │   └── components/
  │       ├── candidates/
  │       ├── AddCandidateDialog.tsx
  │       ├── ImportCandidatesDialog.tsx
  │       ├── Layout.tsx
  │       ├── UI.tsx
  │       └── pages/
  │           ├── Dashboard.tsx
  │           ├── Candidates.tsx
  │           ├── CandidateDetails.tsx
  │           ├── Analytics.tsx
  │           ├── Settings.tsx
  │           ├── App.tsx
  │           └── index.css
```

3. Frontend Files

File	Language	Purpose
App.tsx	TypeScript + React	Main routing and page mapping.
index.css	CSS + Tailwind	Global theme and base styling.
Layout.tsx	TypeScript + React	Sidebar, navigation, mobile menu, header and page shell.
UI.tsx	TypeScript + React	Reusable Button, Card, Input, Progress, StatusBadge and shared UI.
Dashboard.tsx	TypeScript + React	KPIs, enrollment chart, Needs Attention and Recent Enrollments.
Candidates.tsx	TypeScript + React	Search, filters, add/edit/delete, status and import.
CandidateDetails.tsx	TypeScript + React	Profile, credits, applications, progress, reports, MAR, feedback, activity.
Analytics.tsx	TypeScript + React	Program analytics and visual reporting.
Settings.tsx	TypeScript + React	Program/admin configuration.
AddCandidateDialog.tsx	TypeScript + React	Add-candidate form.
ImportCandidatesDialog.tsx	TypeScript + React	Excel/CSV selection and upload UI.
api/candidate.ts	TypeScript	Frontend API functions and backend communication.
api/candidates.js	JavaScript	Candidate-related API/helper file; verify imports before deleting.

4. Backend Folder Structure

```
server/ ■■■■ db.js ■■■■ routes/ ■■■■ candidates.js
```

Backend uses JavaScript. Node.js runs the server, Express provides HTTP routes, and MongoDB stores documents.

File	Language	Purpose
server/db.js	JavaScript	MongoDB connection, database setup and collection access.
server/routes/candidates.js	JavaScript	Candidate APIs and business logic: CRUD, import, status, applications, reports, MAR and

5. Backend Modules

Module	Function
Candidate CRUD	Create, read, update and delete candidates.
Candidate status	Update program status such as Active, Paused, Completed and Expiring Soon.
Credits	Calculate/store total, used and remaining credits based on plan/application activity.
Application history	Store daily application records including date, applications and Updated By.
Excel/CSV import	Multer receives files; XLSX reads spreadsheet data; rows are validated and duplicates handled.
Reports	Candidate-specific report upload/retrieval.
MAR	Candidate-specific Market Analysis Report upload/retrieval.
Activity	Candidate-related activity/history endpoint.
Single candidate	Retrieve one candidate by candidate ID.

6. Main API Routes

```
GET /api/candidates POST /api/candidates POST /api/candidates/import GET /api/candidates/:candidateId
PUT /api/candidates/:candidateId PATCH /api/candidates/:candidateId/status DELETE
/api/candidates/:candidateId GET/POST /api/candidates/:candidateId/applications GET/POST
/api/candidates/:candidateId/reports GET/POST /api/candidates/:candidateId/mar GET
/api/candidates/:candidateId/activity
```

GET = read, POST = create, PUT/PATCH = update, DELETE = remove.

7. End-to-End Flows

Add Candidate

AddCandidateDialog.tsx → candidate.ts → POST /api/candidates → Express → validation/business logic → MongoDB → response → Candidates.tsx

Excel/CSV Import

ImportCandidatesDialog.tsx → FormData → POST /api/candidates/import → Multer → XLSX parsing → validation → duplicate check → MongoDB → result

Dashboard

Dashboard.tsx → getCandidates() → GET /api/candidates → Express → MongoDB → candidate data → React calculations → cards/chart/table

Applications

Candidate Details → application API → POST applications → separate history record → credit/application totals → refreshed UI

8. Main Data Areas

Candidate ■■■ id / name / email ■■■ domain / target role / experience ■■■ plan ■■■ creditsTotal / creditsRemaining / creditsUsed ■■■ startDate / endDate ■■■ status ■■■ notes ■■■ application history ■■■ monthly progress ■■■ reports ■■■ MAR ■■■ feedback ■■■ activity

9. What to Learn — Frontend

Order	Topic	Used in
1	HTML + JSX	All .tsx UI files
2	CSS	index.css and styling
3	JavaScript	Core React/API logic
4	TypeScript	Main .ts/.tsx frontend files
5	React	Components, props, state and hooks
6	Tailwind CSS	Layout and visual styling
7	React Router	App.tsx, Layout.tsx and navigation
8	Fetch + async/await	API functions and page calls
9	Recharts	Dashboard and analytics

10. What to Learn — Backend

Order	Topic	Used in
1	JavaScript	Backend files
2	Node.js	Server runtime
3	Express.js	API routes and middleware
4	REST APIs	GET/POST/PUT/PATCH/DELETE
5	MongoDB	Database and documents
6	Async/await	Database/API operations
7	Multer	File uploads
8	XLSX	Excel/CSV processing

11. Quick File Cheat Sheet

File/type	Meaning
.tsx	TypeScript + JSX; normally React UI.
.ts	TypeScript without JSX; often API/helper logic.
.js	JavaScript; current backend files use this.
.css	CSS; global styling/theme.
React	Builds frontend UI.
Tailwind	Styles frontend UI.
Express	Builds backend HTTP APIs.
MongoDB	Stores application data.
Multer	Handles uploaded files.
XLSX	Reads Excel/spreadsheet files.

12. Easy Mental Model

React renders the UI → TypeScript organizes frontend code → API functions send requests → Express receives requests → backend logic validates/processes → MongoDB stores data → response returns to React.

Current UI theme: #121315 background · #5EB586 primary green · #FFFFFF white · #1A1C1E cards · #9A9DA1 muted text · #303335 borders.

This guide reflects the project structure and functionality discussed/shared so far. Files not provided or confirmed may contain additional modules.